Major Project Report

UFOS

Project Category: University Based Project

*Projexa Team Id – **26E3212***

Submitted in partial fulfilment of the requirement of the degree of

BACHELOR OF Science (CS/DS/Cyber)

Semester-IV

to

K.R Mangalam University

by

Naman Katewa (2401830002)
Sneha Yadav (2401840003)
Nikita Saharan (2401840008)
Kartikeya Drall (2401720014)
Akshaya Tanwar (2401840006)

Under the supervision of

Dr. Rishika Mehta
<Internal Mentor>
Assistant Professor

Department of Computer Science and Engineering
School of Engineering and Technology
K.R Mangalam University, Gurugram- 122001, India

May 2026

DECLARATION

We hereby declare that this project report is our original work and has not been submitted elsewhere. The information presented here is true and correct to the best of my knowledge.

Naman Katewa
2401830002
14/04/2026

Kartikeya Drall
2401720014
14/04/2026

Sneha Yadav
2401840003
14/04/2026

Akshaya Tanwar
2401840006
14/04/2026

Nikita Saharan
2401840008
14/04/2026

CERTIFICATE

This is to certify that the Project Synopsis entitled, "**UFOS**" submitted by "**Naman Katewa(2401830002), Sneha Yadav(2401840003), Kartikeya Drall(2401720014), Akshaya Tanwar(2401840006) and Nikita Saharan(2401840008)"** to **K.R Mangalam University, Gurugram, India,** is a record of Bonafide project work carried out by them under my supervision and guidance and is worthy of consideration for the partial fulfilment of the degree of **Bachelor of Science** in **Computer Science, Data Science and Cybersecurity** of the University.

Dr. Rishika Mehta

Assistant Professor

Date: 20th April 2026

INDEX

S No	Content	Page No
1.	Project Completion Certificate	3
2.	Abstract	5
3.	Introduction	7
4.	Motivation	11
5.	Literature Review/Comparative work evaluation	13
6.	Gap Analysis	16
7.	Problem Statement	17
8.	Objectives	18
9.	Methodology	19
10.	Tools/platform Used	24
11.	Environmental Setup	26
12.	Evaluation Metrics	
13.	Results And Discussion	
14.	Conclusion & Future Work	
15.	References	
16.	Annexure I: Plagiarism Report	
17.	Annexure II Complete implementation code (optional)	
18.	Annexure III Research Paper (Published/Submitted)	

ABSTRACT

The transportation logistics of large academic institutions in the National Capital Region (NCR) present a unique set of operational challenges, particularly for universities with a catchment area that spans across diverse geographic terrains such as Sohna, Gurugram, Faridabad, Palwal, Noida, and Bhiwadi. Traditional methods of fleet management currently employed are largely manual and static, relying on historical precedence and human intuition rather than data-driven decision-making. This lack of optimization leads to severe inefficiencies, including overlapping routes, uneven capacity utilization where some buses are overcrowded while others run empty, and excessive fuel consumption. Furthermore, the reliance on manual planning makes it nearly impossible to dynamically adapt to shifting student demographics each semester, resulting in a rigid system that incurs high operational costs and leaves a significant carbon footprint.

To address these critical inefficiencies, this project proposes the design and implementation of the "University Fleet Optimization System," a robust, zero-operational-cost routing solution. Unlike commercial fleet management tools that impose heavy financial burdens through recurring licensing fees or pay-per-request API costs (such as Google Maps Platform), this system is built entirely on a self-hosted, open-source technology stack. By leveraging the advanced Vehicle Routing Problem (VRP) algorithms provided by the Google OR-Tools library, the system mathematically models the entire transport network. It intelligently clusters students based on their geographic location and generates optimal route sequences that minimize the total fleet distance traveled, ensuring that every student from distant nodes like Bhiwadi to central hubs like Gurugram is served efficiently.

The technical architecture of the proposed solution is designed for autonomy and sustainability. It features a self-hosted instance of the Open Source Routing Machine (OSRM) running via Docker, which enables the calculation of precise travel time matrices across the extended NCR road network without any external dependencies or costs. This backend is coupled with a high-performance Python (FastAPI) application logic layer and a modern, responsive Next.js web dashboard for administration. This architecture allows the university to visualize stops, manage demand, and simulate "what-if" scenarios for future planning. By strictly adhering to hard constraints—such as a maximum ride time of 2 hours and specific bus capacity limits—the system guarantees a balance between operational efficiency and student comfort. Ultimately, this project demonstrates how deep-tech optimization can be democratized and deployed cost-effectively to solve complex, real-world logistical problems in the education sector.

KEYWORDS: Vehicle Routing Problem (VRP), Fleet Management, Route Optimization, Open Source, Smart Transportation, OSRM, Google OR-Tools

Chapter 1
Introduction

1. Background of the project

The rapid expansion of educational institutions and the increasing geographic dispersion of student populations have fundamentally transformed the logistical requirements of university transportation. In sprawling urban and semi-urban agglomerations such as the extended National Capital Region (NCR)—encompassing Sohna, Gurugram, Faridabad, Palwal, Noida, and Bhiwadi—managing a daily commute for thousands of students is a highly complex operational challenge. Traditionally, institutional fleet management has relied on manual planning, historical precedence, and static spreadsheet-based scheduling. However, as student demographics shift and traffic congestion worsens, these manual methodologies result in severe inefficiencies, including overlapping routes, uneven vehicle utilization (overcrowding versus empty seats), extended commute times, and inflated fuel consumption.

Furthermore, the economic and environmental costs of inefficient transportation are becoming unsustainable. Rising fuel prices heavily impact institutional budgets, while unnecessary "deadhead" (empty) kilometers contribute significantly to carbon emissions. To combat this, the emergence of algorithmic route optimization and fleet management systems has become a mainstream operational necessity globally. A data-driven routing system can dramatically increase the efficiency of a transport fleet, ensuring that vehicles are packed optimally and reach their destinations within strict time constraints. Once an automated scheduling system is implemented, transport

administrators no longer need to rely on guesswork to accommodate new students or adjust to shifting regional demands.

While the importance of optimized routing is well established, the barrier to entry for educational institutions remains the prohibitive cost of commercial software. Modern fleet management often relies on proprietary solutions (Commercial SaaS) or expensive per-request mapping APIs (such as Google Maps Platform). These recurring licensing fees can offset the savings generated by fuel reduction. Therefore, there is a critical need for a "Zero-Operational Cost" alternative. By leveraging open-source geospatial data (OpenStreetMap) and self-hosted routing engines (OSRM), combined with advanced mathematical solvers (Google OR-Tools), it is now possible to build a highly sophisticated, bespoke fleet management system. Such a system not only automates the Multi-Depot Capacitated Vehicle Routing Problem with Time Windows (MD-CVRPTW) but does so without incurring recurring software costs, providing administrators with a powerful, accessible web dashboard to manage operations efficiently.

Table 1. Existing systems

Factors	***Evaluation Criteria***	***Commercial Fleet SaaS***	***Manual Spreadsheet Planner***	***UFOS***
Cost & Licensing	- Upfront costs (installation, setup)	Moderate	Low	Moderate

	- Recurring Operational Costs	Very high	Zero	Zero
Optimization Capabilities	- Algorithmic route generation (VRP)	Yes (Proprietary)	No	Yes (Christofides & Simulated Annealing)
	- Custom Constraints	Limited	Manual Enforcement	Highly Customizable
	- Split Delivery for high demand nodes	Rarely Supported	Impossible to scale	Yes
Geographic Adaptability	- Custom physics and vehicle profiles	Limited	None	Custom bus profile
	- Offline Capability	No	Yes	Yes (Self Hosted)
Visualization & Performance	- Browser rendering of	Moderate	N/A	High (PM Tiles & Leaflet)

	massive route data			
	- Scenarios Comparison (Strict vs Suggested)	Rarely Supported	Very Slow	Seamless & Built-in
Data Privacy & Control	- Storage Options	Cloud-based	Local	Local / Cloud
	- Compliance with institutional data privacy	Moderate	High	High
Scalability	- Ability to scale for additional buses/stops	High (costs increase)	Very Low	Highly Scalabe (No cost increase)

2. Motivation

In recent years, the logistical burden on universities operating in the extended NCR has grown exponentially. The catchment area for K.R. Mangalam University alone spans across highly diverse topographies, from the dense urban traffic of Gurugram and Noida to the distant, semi-urban nodes of Palwal and Bhiwadi. Manually routing a fleet of 40+ buses to accommodate over 3,000 students across these regions creates a combinatorial explosion of

possibilities. It is mathematically impossible for a human planner to consistently find the most efficient routes while simultaneously balancing strict bus capacities and a mandatory campus arrival deadline. This often leads to compromised student experiences, where some endure rides exceeding two hours, while the institution suffers from inflated fuel and maintenance budgets.

Even though the theoretical solutions to these problems—such as the Vehicle Routing Problem (VRP)—have existed in operations research for decades, their practical application has been gatekept by commercial entities. Market research indicates that enterprise-grade fleet management software and routing APIs charge exorbitant recurring fees, draining educational resources. We are motivated by the conviction that deep-tech optimization should be democratized. By harnessing the power of open-source tools, we can mitigate the financial risks associated with digital transformation in the education sector. If transport administrators are equipped with a "Zero-Cost" system that automatically handles the complex mathematics of routing, they can shift their focus from manual data entry to strategic operational improvements.

The core motivation of the "University Fleet Optimization System" (UFOS) is to completely eliminate software licensing costs while outperforming generic routing tools. By engineering a custom Open Source Routing Machine (OSRM) instance calibrated specifically for heavy-bus physics (incorporating strict U-turn penalties and urban speed caps), the system guarantees that mathematically generated routes are practically drivable.

Furthermore, the integration of advanced metaheuristics—specifically pairing the "Christofides" algorithm with "Simulated Annealing"—allows the system to escape local minima and pack buses with unprecedented efficiency. The motivation culminates in providing a seamless, high-performance Next.js dashboard utilizing PMTiles, ensuring that the transport department can visualize thousands of coordinates, simulate dynamic split-delivery scenarios, and confidently deploy routes that save 10-15% in fuel consumption while maintaining 100% student coverage.

Chapter 2
LITERATURE REVIEW

1. Review of existing literature

TRADITIONAL FLEET MANAGEMENT VS. ALGORITHMIC OPTIMIZATION:

There has been extensive research on the value of algorithmic optimization for logistics and supply chain management, but its application in educational institutional transport remains limited. Traditionally, university fleet routing relies on manual planning, historical precedence, and static spreadsheet models. Studies on urban transit indicate that static routing leads to severe inefficiencies, including overlapping paths, uneven passenger distribution, and inflated fuel consumption. The transition from manual planning to algorithmic optimization—specifically using operations research methods—has been proven to reduce operational costs by up to 20%. However, the literature highlights a significant barrier: the high recurring cost of commercial SaaS platforms and proprietary mapping APIs (such as Google Maps Platform), which often deter educational institutions from adopting these necessary technological upgrades.

THE VEHICLE ROUTING PROBLEM (VRP) AND MD-CVRPTW:

The core of modern fleet optimization lies in the Vehicle Routing Problem (VRP). For a university scenario involving multiple parking locations, strict seating limits, and campus arrival deadlines, the problem is modeled as the Multi-Depot Capacitated Vehicle Routing Problem with Time Windows (MD-CVRPTW). Extensive literature in operations research demonstrates that basic algorithms (like the Clarke-Wright "Savings" heuristic) often get trapped in

local minima when dealing with complex, large-scale datasets. Recent advancements suggest that pairing graph theory heuristics (such as "Christofides") with metaheuristics (like "Simulated Annealing" or "Guided Local Search") yields vastly superior results, drastically reducing the required fleet size and total distance traveled while strictly adhering to capacity and time constraints.

OPEN-SOURCE GEOSPATIAL TECHNOLOGIES (OSRM):

The reliance on commercial distance matrix APIs is a primary cost driver in modern routing applications. In existing research regarding zero-cost logistics, the Open Source Routing Machine (OSRM) emerges as a powerful alternative. OSRM, a C++ routing engine designed to run on OpenStreetMap (OSM) data, allows for self-hosted matrix calculations. Crucially, literature on heavy-vehicle routing emphasizes that generic "car" physics cannot be applied to 12-meter buses. Customizing the routing engine's Lua profiles to enforce heavy-bus physics—such as strict U-turn penalties and urban speed caps—is highly documented as a necessity to ensure that mathematically generated routes are practically drivable in real-world urban environments like the NCR.

HIGH-PERFORMANCE VISUALIZATION IN LOGISTICS:

Visualizing complex logistics data on web platforms presents a significant technical challenge. Traditional mapping libraries often crash or suffer from severe browser lag when attempting to render thousands of coordinate points (polylines) simultaneously. Modern literature in web GIS (Geographic Information Systems) proposes the shift from traditional GeoJSON rendering to edge-stored vector tiles. Technologies like PMTiles combined with modern

frameworks like Next.js allow for the seamless, crash-free rendering of massive route geometries. This ensures that transport administrators can view the entire optimized network across a region without performance degradation.

Table 2. LITERATURE REVIEW/COMPARITIVE WORK

Project/System Type	***Objectives***	***Technologies Used***	***Outcomes and Findings***
Commercial Fleet SaaS (e.g., Samsara, Routific)	Optimize delivery and transport routes for enterprises	Proprietary algorithms, Cloud-based APIs (Google/Mapbox)	High accuracy and reliability, but prohibitively high recurring monthly/per-asset licensing fees.
Traditional University Transport Systems	Manage daily student commutes	Spreadsheets, Manual intuition, Static bus assignments	Highly inefficient, overlapping routes, massive fuel waste, and unequal passenger loads.
Basic VRP Academic Implementations	Solve theoretical routing problems	Basic "Savings" heuristics, standard distance matrices	Often stuck in local minima; ignores real-world heavy vehicle physics (U-turns, traffic).
Cloud-Reliant Logistics Dashboards	Visualize fleet movements	Standard React, GeoJSON rendering	Prone to browser crashes and lag when rendering thousands of route coordinates simultaneously.

Proposed System (UFOS)	**Zero-cost MD-CVRPTW optimization for Extended NCR**	**Google OR-Tools, Self-Hosted OSRM, Next.js, PMTiles**	**Fuel reduction, zero recurring costs, custom bus physics, crash-free high-performance rendering.**

2. GAP ANALYSIS

From the numerous researches and products available for fleet management and VRP optimization, the resultant software is almost always a commercial, proprietary product aimed at enterprise logistics and delivery companies. These systems charge exorbitant recurring API fees, making them inaccessible for educational institutions operating on fixed budgets. Furthermore, many academic or open-source implementations of VRP algorithms rely on basic heuristics that fail to account for real-world heavy vehicle physics—such as the inability of a 50-seater bus to easily make U-turns in dense areas like Gurugram or Faridabad.

Our project bridges this critical gap. UFOS provides an enterprise-grade, algorithmically superior routing solution that operates with a strictly "Zero-Operational Cost" architecture. By utilizing a self-hosted OSRM with a custom Lua bus-physics profile, and pairing the Christofides heuristic with Simulated Annealing, the system guarantees highly efficient, practically drivable routes without relying on paid APIs. Additionally, unlike systems that struggle with rendering large datasets, UFOS integrates PMTiles within a Next.js environment, offering a robust, lag-free dashboard. Hence, we have covered a unique subset of requirements: deep-tech mathematical optimization,

realistic road physics, and high-performance UI, all delivered at zero recurring cost.

3. PROBLEM STATEMENT

Managing university transportation across a massive and diverse catchment area—spanning Sohna, Gurugram, Faridabad, Palwal, Noida, and Bhiwadi—is a highly complex, manually intensive task. Currently, the routing of 40+ buses for over 3,000 students relies on static, fixed routes that lead to severe inefficiencies, inflated fuel costs, and uneven student loads.

It is mathematically impossible for a human planner to consistently calculate the shortest possible paths while simultaneously enforcing strict constraints, such as a maximum 2-hour ride time and strict 50-seat bus capacities. Furthermore, existing commercial software capable of solving this Multi-Depot Capacitated Vehicle Routing Problem (MD-CVRPTW) relies on paid mapping APIs (like Google Maps), introducing prohibitive recurring costs. This creates a critical need for a zero-cost, intelligent fleet optimization system that can automate route planning, dynamically split high-demand stops, and mathematically minimize total fleet mileage without incurring ongoing software licensing fees.

4. OBJECTIVES

The features of the proposed system include:

1. **Optimize** – Mathematically minimizes total fleet mileage using the MD-CVRPTW algorithm (Christofides + Simulated Annealing).
2. **Eliminate Costs** – Replaces expensive commercial APIs with a self-hosted Open Source Routing Machine (OSRM) for zero recurring operational expenses.
3. **Enforce Constraints** – Strictly adheres to maximum ride times (e.g., 120 minutes), bus seating capacities, and campus arrival deadlines.
4. **Physics-Aware Routing** – Utilizes custom Lua profiles to enforce heavy-bus driving physics, preventing mathematically impossible U-turns in urban traffic.
5. **Visualize Seamlessly** – Renders massive datasets and complex route geometries without browser lag using a Next.js dashboard and PMTiles.

The objective is to overcome the gap left by expensive commercial logistics tools and inefficient manual planning, creating a smart, automated fleet optimization system for the university. The optimized routes can be seamlessly exported for drivers, reducing fuel consumption by 10-15%. The use of Google OR-Tools makes the system mathematically rigorous, while the custom Next.js frontend ensures it remains highly accessible and easy to understand for transport administrators.

CHAPTER 3: METHODOLOGY

The methodology section of this project outlines the systematic procedures, algorithms, and technical frameworks utilized to develop the University Fleet Optimization System (UFOS). It details the flow of data, the architectural design, and the step-by-step development lifecycle required to solve the Multi-Depot Capacitated Vehicle Routing Problem with Time Windows (MD-CVRPTW).

3.1 **Overall architecture /Flow chart** :

The overall architecture of UFOS is designed as a decoupled, high-performance web system that integrates front-end visualization, an asynchronous backend, and heavy-duty mathematical and routing engines. The system operates through a clear pipeline, transforming raw administrative data into optimized, physics-aware bus routes.

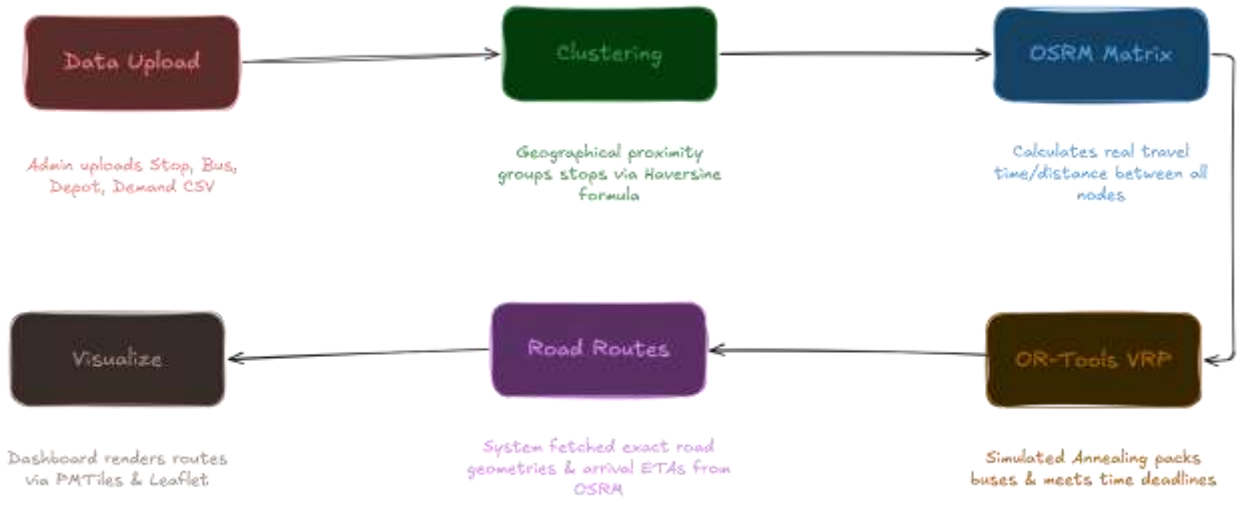


Figure 1. Project Flow Diagram

As illustrated in the Project Flow Diagram, the process begins with Data Upload, where the transport admin uploads CSV files containing Stop, Bus, Depot, and Demand data. This data undergoes Clustering, where geographical proximity groups stops using the Haversine formula. The clustered nodes are then passed to the OSRM Matrix engine, which calculates the real-world travel time and distance between all nodes. This massive cost matrix is fed into the OR-Tools VRP core, utilizing Simulated Annealing to pack buses and meet time deadlines. The output is a set of Road Routes, where the system fetches the exact road geometries and ETAs from OSRM. Finally, the Visualize module renders these routes on the dashboard via PMTiles and Leaflet.

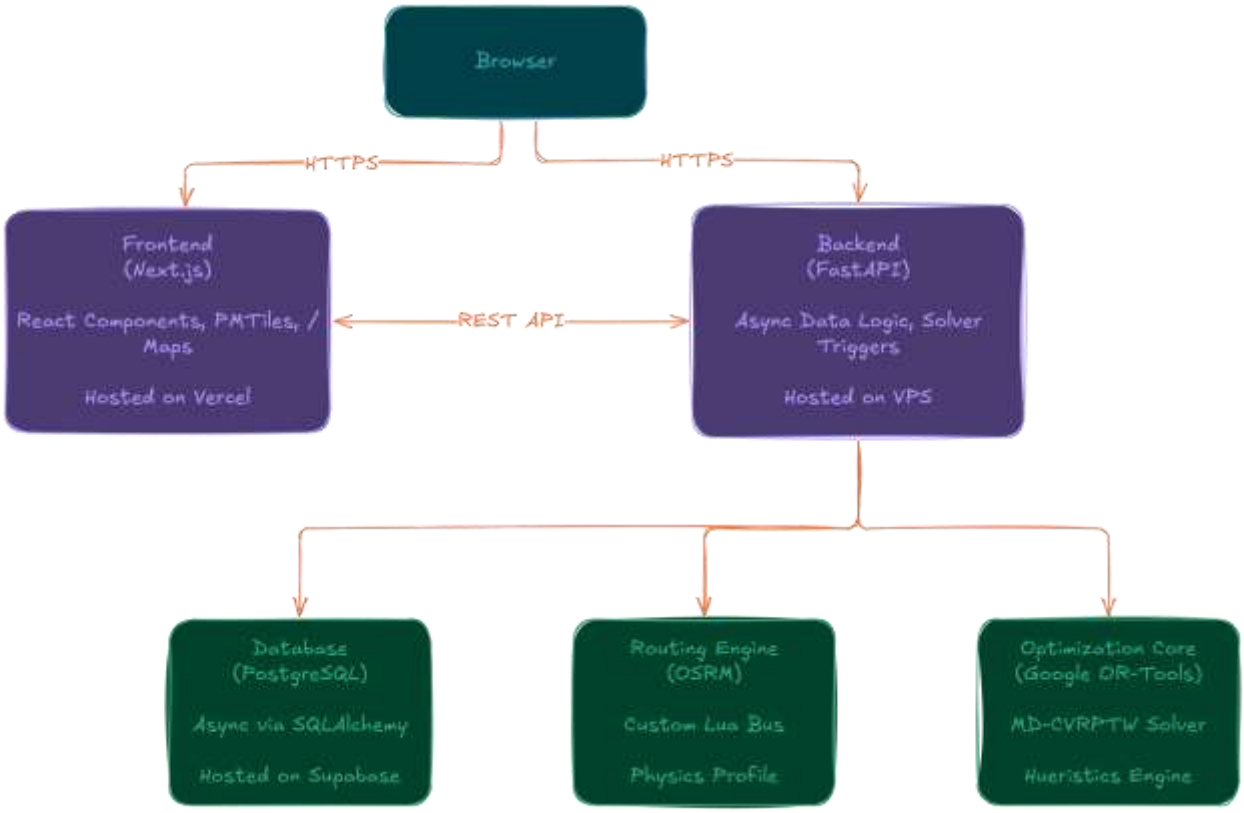


Figure 2. Architecture Diagram

The Technical Architecture demonstrates a modern web stack. The Browser communicates via HTTPS to the Frontend (Next.js) hosted on Vercel, which handles React components and PMTiles rendering. The frontend communicates via a REST API to the Backend (FastAPI) hosted on a VPS. The backend manages asynchronous data logic and acts as the trigger for the core engines. It interfaces with three primary microservices: the Database (PostgreSQL) via SQLAlchemy for storage, the Routing Engine (OSRM) running a custom Lua bus physics profile, and the Optimization Core (Google OR-Tools) housing the MD-CVRPTW solver and heuristic engines.

3.2 Data Description

Data Source: The project relies on two primary data sources. Map data is sourced from OpenStreetMap (OSM) extracts for North India, downloaded via Geofabrik (india-latest.osm.pbf). The student demand and stop locations are sourced from internal university transport logs, providing geotagged coordinates for pickup points across Palwal, Faridabad, Noida, Gurugram, and Bhiwadi.

Data Collection Process: Administrative data is collected via CSV uploads through the UFOS dashboard. The geographic data is pre-downloaded from open-source repositories and loaded into the local OSRM container.

Data Type: The system handles geospatial data (latitude and longitude coordinates), numerical data (student demand counts, bus seating capacities, maximum ride times in minutes), and categorical data (bus types, depot locations).

Data Size: The system processes substantial datasets, including over 3,000 student stops, 100+ buses, and a massive OSM extract (approx. 2.5 GB) representing the entire road network of the extended NCR region.

Data Format: Input data is formatted as CSV files. Map data utilizes the compressed .osm.pbf format. Communication between the frontend, backend, and OR-Tools solver utilizes structured JSON payloads.

Data Preprocessing: Raw student coordinates undergo preprocessing via the Haversine formula to cluster highly dense neighborhood stops into single logistical nodes. The raw .osm.pbf map data is preprocessed by the OSRM backend to extract, partition, and customize a routing graph specific to bus physics.

Data Quality Assurance: To ensure the generated routes are practically drivable, custom Lua profiles are applied during the map data processing. This enforces strict 40-second U-turn penalties and caps speeds to realistic urban traffic limits, guaranteeing the integrity of the travel-time data.

Data Variables: Key variables include stop_id, latitude, longitude, student_demand, bus_id, capacity, and time_window (arrival deadlines).

3.3 Exploratory data Analysis

Exploratory Data Analysis (EDA) in UFOS focuses on geographic distribution and logistical constraints rather than traditional statistical modeling:

- **Geographical Distribution:** Visualizing the scatter of student stops across the NCR to identify high-density zones (e.g., Faridabad) versus sparse outliers (e.g., Bhiwadi). This dictates the necessity of split-delivery logic for dense areas.
- **Distance Matrix Analysis:** Analyzing the OSRM-generated Time/Distance matrix to identify "isolated nodes"—stops that lack proper road connections in the OSM data, requiring manual coordinate correction.
- **Demand Profiling:** Calculating summary statistics on student demand per stop to set baseline capacity constraints for the OR-Tools solver.

3.4 Procedure /Development Life Cycle

The development life cycle of UFOS was executed in four distinct phases to ensure mathematical accuracy and system stability:

Phase 1 - Data & Physics: Focus was placed on the foundational routing environment. We engineered a custom Lua profile for OSRM to ensure mathematically generated routes are actually drivable by heavy 12-meter vehicles, replacing generic car physics with strict U-turn penalties.

Phase 2 - Algorithmic Tuning: We conducted exhaustive benchmarking of over 30 Google OR-Tools configurations. During this phase, it was identified that basic "Savings" heuristics got stuck in local minima, leading to massive logistical inefficiencies.

Phase 3 - The Breakthrough: The optimal mathematical configuration was discovered and implemented: pairing the "Christofides" graph theory heuristic (for initial route generation) with a "Simulated Annealing" metaheuristic (for local search). This successfully modeled the MD-CVRPTW, processing datasets of 3000+ students and significantly reducing the required fleet size.

Phase 4 - Production & Security: The focus shifted to deployment and high-performance visualization. We overcame browser lag and crashing by implementing PMTiles for edge-stored map rendering. Finally, strict browser CORS policies and Next.js caching errors were resolved by engineering a reverse-proxy tunnel using Nginx and Let's Encrypt SSL certificates.

1. Details of tools, software, and equipment utilized.

PLATFORM USED

For this project, we have used various state-of-the-art technologies chosen for their performance in heavy mathematical computation and web visualization.

PROGRAMMING LANGUAGES: PYTHON & TYPESCRIPT

We utilized Python for the backend architecture. Python is a widely used general-purpose, high-level programming language that excels in data processing and algorithmic implementations. It allows seamless integration with heavy C++ libraries like Google OR-Tools and provides rapid API development through the FastAPI framework.

Reasons for Selecting Python (Backend):

- Extensive support for mathematical optimization libraries (OR-Tools).
- High-performance asynchronous capabilities via FastAPI.
- Seamless integration with data manipulation libraries (Pandas, NumPy).

We utilized TypeScript (superset of JavaScript) for the frontend architecture via the Next.js framework.

Reasons for Selecting TypeScript (Frontend):

- Provides strict type safety, reducing runtime errors when handling complex GeoJSON data.
- Industry-standard for modern React applications.
- Excellent integration with mapping libraries like Leaflet.js.

CORE TECHNOLOGIES:

1. **Google OR-Tools:** An open-source software suite for optimization, used to solve the core Multi-Depot Capacitated Vehicle Routing Problem with Time Windows (MD-CVRPTW).

2. **OSRM (Open Source Routing Machine):** A high-performance routing engine written in C++ designed to run on OpenStreetMap data, self-hosted via Docker to eliminate API costs.

3. **FastAPI:** A modern, fast web framework for building APIs with Python based on standard Python type hints.

4. **Next.js & PMTiles:** A React framework paired with cloud-optimized archives of map tiles, allowing the browser to render thousands of route coordinates without crashing.
5. **PostgreSQL & SQLAlchemy:** A powerful, open-source object-relational database system used for structured data storage, accessed asynchronously via SQLAlchemy.

2. ENVIRONMENTAL SETUP

SOFTWARE REQUIREMENTS

Below are the requirements to run this software:

- **Operating System:** Linux / Windows / macOS (Cross-platform compatibility).
- **Containerization:** Docker Desktop and Docker Compose (Crucial for hosting the OSRM routing engine).
- **Backend Environment:** Python 3.10+, pip package manager.
- **Frontend Environment:** Node.js (v18+), npm or pnpm package manager.
- **Database:** PostgreSQL server.
- **Python Packages:** fastapi, uvicorn, ortools, sqlalchemy, pandas, requests.
- **NPM Packages:** next, react-leaflet, pmtiles, @tanstack/react-query.

HARDWARE REQUIREMENTS

To successfully build and run the optimization matrix, the following hardware is required:

- **Processor:** Modern multi-core CPU (Intel i5/i7 or AMD Ryzen equivalent).
- **RAM:** Minimum 8GB RAM (16GB+ strongly recommended for processing the 2.5GB india-latest.osm.pbf file during OSRM graph extraction).
- **Storage:** At least 20GB of free SSD storage for map data and Docker images.
- **Network:** Active internet connection for fetching map tiles and initial data downloads.

PLATFORMS ALREADY TESTED ON:

The system has been successfully compiled, tested, and deployed on Windows 10/11, Ubuntu Linux (VPS Hosting), and macOS environments.

Chapter 4
Implementation

4.1 Detailed explanation of how the project was implemented

The implementation of the University Fleet Optimization System (UFOS) followed a decoupled, microservices-oriented approach, ensuring that the heavy computational tasks were isolated from the user interface. The project was implemented in four distinct functional layers:

1. **Geospatial Data Processing & Routing Engine (OSRM):** The foundation of the implementation involved self-hosting the Open Source Routing Machine (OSRM). Instead of relying on paid APIs, the raw North India map data (india-latest.osm.pbf) from Geofabrik was downloaded and compiled locally. The engine was containerized using Docker. To ensure realistic routing for heavy vehicles, the default car physics profile was overwritten with a custom Lua script that enforced bus-specific constraints (e.g., lower speed limits and heavy U-turn penalties). The osrm-extract, osrm-partition, and osrm-customize pipelines were executed to generate a highly optimized travel-time graph.

2. **Backend Logic & API (FastAPI):** The backend was developed using Python's FastAPI framework due to its high performance and asynchronous capabilities. This layer acts as the orchestrator. It receives student demand and stop coordinates from the database (PostgreSQL via SQLAlchemy), queries the OSRM container to build a massive distance/time matrix, and formats this data for the optimization solver.

3. **Optimization Core (Google OR-Tools):** The mathematical core of the project was implemented using the Google OR-Tools library. The problem was modeled as a Multi-Depot Capacitated Vehicle Routing Problem with Time Windows (MD-CVRPTW). The solver was configured to ingest the OSRM time matrix, bus capacities (e.g., 50 seats), and the strict campus arrival deadline.

4. **Frontend & Visualization (Next.js):** The administrative dashboard was built using Next.js and Tailwind CSS. To handle the visualization of thousands of geographical coordinates without crashing the browser, the frontend was integrated with PMTiles and Leaflet.js. The frontend communicates with the FastAPI backend via RESTful endpoints, utilizing TanStack Query for state management and caching.

4.2 Description of algorithms, code snippets, or design diagrams

Algorithm 1: Distance Calculation and Stop Clustering Before routing, nearby student stops were clustered to reduce the computational load on the solver. The Haversine formula was

implemented to calculate the great-circle distance between two points on a sphere from their longitudes and latitudes.

$$d = 2r * \arcsin \quad sqrt\left(\sin^{2\left(\frac{phi_2 - phi_1}{2}\right)} + \cos(phi_1) * \cos(phi_2)\left(\sin^{2\left(\frac{lambda_2 - lambda_1}{2}\right)} *\right)\right))$$

Algorithm 2: MD-CVRPTW Optimization Pipeline The routing optimization was implemented using a two-step algorithmic approach within OR-Tools:

1. **Initial Solution Strategy (Christofides Heuristic):** Used to find a fast, mathematically sound initial route across the NCR nodes.
2. **Local Search Metaheuristic (Simulated Annealing):** Applied to the initial solution to escape local minima, iteratively swapping nodes between buses to find the absolute minimum fleet mileage while strictly adhering to the 120-minute maximum ride time.

Code Snippet 1: OSRM Custom Lua Profile (Bus Physics)

To prevent the algorithm from routing 12-meter buses down narrow alleys or suggesting impossible U-turns, the bus.lua profile was modified before compiling the OSRM graph:

```
api_version = 4

Set = require('lib/set')
Sequence = require('lib/sequence')
```

```
Handlers = require("lib/way_handlers")
Relations = require("lib/relations")
Obstacles = require("lib/obstacles")
find_access_tag = require("lib/access").find_access_tag
limit = require("lib/maxspeed").limit
Utils = require("lib/utils")
Measure = require("lib/measure")

function setup()
  return {
    properties = {
      max_speed_for_map_matching      = 80/3.6,
      weight_name                     = 'routability',
      process_call_tagless_node      = false,
       u_turn_penalty                   = 40,    -- Buses can't U-turn
easily
      continue_straight_at_waypoint  = true,
      use_turn_restrictions          = true,
      left_hand_driving              = false,
    },

    default_mode              = mode.driving,
    default_speed             = 10,
    oneway_handling           = true,
    side_road_multiplier      = 0.8,
    turn_penalty               = 12,  -- Higher than car (7.5) — buses
turn slower
    speed_reduction           = 0.8,
    turn_bias                 = 1.075,
    cardinal_directions       = false,

    -- Bus dimensions
    vehicle_height = 3.5,    -- meters
    vehicle_width  = 2.55,   -- meters
    vehicle_length = 12.0,   -- meters (full-size bus)
```

```
    vehicle_weight = 15000,  -- kg

    suffix_list = {
       'N', 'NE', 'E', 'SE', 'S', 'SW', 'W', 'NW', 'North', 'South',
'West', 'East', 'Nor', 'Sou', 'We', 'Ea'
    },

    barrier_whitelist = Set {
      'cattle_grid',
      'border_control',
      'toll_booth',
      'sally_port',
      'no',
      'entrance',
      'height_restrictor',
      'arch'
    },

     -- Buses ARE allowed on bus/psv roads (unlike car profile which
blacklists them)
    access_tag_whitelist = Set {
      'yes',
      'motorcar',
      'motor_vehicle',
      'vehicle',
      'permissive',
      'designated',
      'hov',
      'bus',
      'psv'
    },

    access_tag_blacklist = Set {
      'no',
      'agricultural',
```

```
  'forestry',
  'emergency',
  'customers',
  'private',
  'delivery',
  'destination'
},

service_access_tag_blacklist = Set {
    'private'
},

restricted_access_tag_list = Set {
  'private',
  'delivery',
  'destination',
  'customers',
},

access_tags_hierarchy = Sequence {
  'bus',
  'psv',
  'motorcar',
  'motor_vehicle',
  'vehicle',
  'access'
},

service_tag_forbidden = Set {
  'emergency_access'
},

restrictions = Sequence {
  'bus',
  'psv',
```

```
  'motorcar',
  'motor_vehicle',
  'vehicle'
},

classes = Sequence {
    'toll', 'motorway', 'ferry', 'restricted', 'tunnel'
},

excludable = Sequence {
    Set {'toll'},
    Set {'motorway'},
    Set {'ferry'}
},

avoid = Set {
  'area',
  'reversible',
  'impassable',
  'hov_lanes',
  'steps',
  'construction',
  'proposed'
},

-- Bus-specific speeds (km/h) — lower than car
speeds = Sequence {
  highway = {
    motorway        = 80,
    motorway_link   = 40,
    trunk           = 60,
    trunk_link      = 35,
    primary         = 50,
    primary_link    = 25,
    secondary       = 40,
```

```
    secondary_link  = 20,
    tertiary        = 35,
    tertiary_link   = 20,
    unclassified    = 25,
    residential     = 20,
    living_street   = 10,
    service         = 15
  }
},

-- Heavier penalties for narrow service roads
service_penalties = {
  alley             = 0.3,  -- Buses should strongly avoid alleys
  parking           = 0.3,
  parking_aisle     = 0.3,
  driveway          = 0.3,
  ["drive-through"] = 0.3,
  ["drive-thru"]    = 0.3
},

barrier_penalties = {
  gate      = 60,
  lift_gate = 60,
},

restricted_highway_whitelist = Set {
  'motorway',
  'motorway_link',
  'trunk',
  'trunk_link',
  'primary',
  'primary_link',
  'secondary',
  'secondary_link',
  'tertiary',
```

```
    'tertiary_link',
    'residential',
    'living_street',
    'unclassified',
    'service'
  },

  construction_whitelist = Set {
    'no',
    'widening',
    'minor',
  },

  route_speeds = {
    ferry = 5,
    shuttle_train = 10
  },

  bridge_speeds = {
    movable = 5
  },

  surface_speeds = {
    asphalt = nil,
    concrete = nil,
    ["concrete:plates"] = nil,
    ["concrete:lanes"] = nil,
    paved = nil,

    cement = 60,
    compacted = 60,
    fine_gravel = 60,

    paving_stones = 50,
    metal = 50,
```

```
    bricks = 50,

    grass = 30,
    wood = 30,
    sett = 30,
    grass_paver = 30,
    gravel = 30,
    unpaved = 30,
    ground = 30,
    dirt = 30,
    pebblestone = 30,
    tartan = 30,

    cobblestone = 20,
    clay = 20,

    earth = 10,
    stone = 10,
    rocky = 10,
    sand = 10,

    mud = 5
  },

  tracktype_speeds = {
    grade1 =  40,
    grade2 =  30,
    grade3 =  20,
    grade4 =  15,
    grade5 =  10
  },

  smoothness_speeds = {
    intermediate    =  60,
    bad             =  30,
```

```
      very_bad        =  15,
      horrible        =  5,
      very_horrible   =  0,
      impassable      =  0
    },

    -- Bus maxspeed defaults — lower than car
    maxspeed_table_default = {
      urban = 50,
      rural = 60,
      trunk = 70,
      motorway = 80
    },

    maxspeed_table = {
      ["none"] = 80
    },

    relation_types = Sequence {
      "route"
    },

    highway_turn_classification = {
    },

    access_turn_classification = {
    }
  }
end

function process_node(profile, node, result, relations)
  local access = find_access_tag(node, profile.access_tags_hierarchy)
  if access then
            if     profile.access_tag_blacklist[access]     and     not
profile.restricted_access_tag_list[access] then
```

```
      result.barrier = true
    end
  else
    local barrier = node:get_value_by_key("barrier")
    if barrier then
      local restricted_by_height = false
      if barrier == 'height_restrictor' then
                                    local        maxheight        =
Measure.get_max_height(node:get_value_by_key("maxheight"), node)
               restricted_by_height   =   maxheight   and   maxheight   <
profile.vehicle_height
      end

      local bollard = node:get_value_by_key("bollard")
      local rising_bollard = bollard and "rising" == bollard

      local kerb = node:get_value_by_key("kerb")
      local highway = node:get_value_by_key("highway")
      local flat_kerb = kerb and ("lowered" == kerb or "flush" == kerb)
      local highway_crossing_kerb = barrier == "kerb" and highway and
highway == "crossing"

      local sensory = node:get_value_by_key("sensory")
      local audible_fence = barrier == "fence" and sensory and (sensory
== "audible" or sensory == "audio")

      local barrier_penalty = profile.barrier_penalties[barrier]

      if not profile.barrier_whitelist[barrier]
                and not rising_bollard
                and not flat_kerb
                and not highway_crossing_kerb
                and not audible_fence
                and not barrier_penalty
                or restricted_by_height then
```

```
        result.barrier = true
      end
    end
  end

  Obstacles.process_node(profile, node, result, relations)
end

function process_way(profile, way, result, relations)
  local data = {
    highway = way:get_value_by_key('highway'),
    bridge = way:get_value_by_key('bridge'),
    route = way:get_value_by_key('route')
  }

  if (not data.highway or data.highway == '') and
  (not data.route or data.route == '')
  then
    return
  end

  handlers = Sequence {
    WayHandlers.default_mode,
    WayHandlers.blocked_ways,
    WayHandlers.avoid_ways,
    WayHandlers.handle_height,
    WayHandlers.handle_width,
    WayHandlers.handle_length,
    WayHandlers.handle_weight,
    WayHandlers.access,
    WayHandlers.oneway,
    WayHandlers.destinations,
    WayHandlers.ferries,
    WayHandlers.movables,
    WayHandlers.service,
```

```
    WayHandlers.hov,
    WayHandlers.speed,
    WayHandlers.maxspeed,
    WayHandlers.surface,
    WayHandlers.penalties,
    WayHandlers.classes,
    WayHandlers.turn_lanes,
    WayHandlers.classification,
    WayHandlers.roundabouts,
    WayHandlers.startpoint,
    WayHandlers.driving_side,
    WayHandlers.names,
    WayHandlers.weights,
    WayHandlers.way_classification_for_turn
  }

  WayHandlers.run(profile, way, result, data, handlers, relations)

  if profile.cardinal_directions then
      Relations.process_way_refs(way, relations, result)
  end
end

function process_turn(profile, turn)
  local turn_penalty = profile.turn_penalty
  local turn_bias = turn.is_left_hand_driving and 1. / profile.turn_bias
or profile.turn_bias

  if turn.number_of_roads > 2 or turn.source_mode ~= turn.target_mode
or turn.is_u_turn then
    if turn.angle >= 0 then
      turn.duration = turn.duration + turn_penalty / (1 + math.exp( -
((13 / turn_bias) *  turn.angle/180 - 6.5*turn_bias)))
    else
```

```
      turn.duration = turn.duration + turn_penalty / (1 + math.exp( -
((13 * turn_bias) * -turn.angle/180 - 6.5/turn_bias)))
    end

    if turn.is_u_turn then
      turn.duration = turn.duration + profile.properties.u_turn_penalty
    end
  end

  if profile.properties.weight_name == 'distance' then
     turn.weight = 0
  else
     turn.weight = turn.duration
  end

  if profile.properties.weight_name == 'routability' then
      if not turn.source_restricted and turn.target_restricted then
          turn.weight = constants.max_turn_weight
      end
  end
end

return {
  setup = setup,
  process_way = process_way,
  process_node = process_node,
  process_turn = process_turn
}
```

Code Snippet 2: OR-Tools Routing Model Initialization (Python)

```
"""OR-Tools CVRPTW optimizer service for bus route optimization"""

import asyncio
```

```python
import logging
import time
import traceback
import numpy as np
from uuid import UUID
from typing import Tuple, List, Dict, Any
from fastapi import HTTPException
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload
from database import async_session_maker
from ortools.constraint_solver import routing_enums_pb2
from ortools.constraint_solver import pywrapcp

from config import get_settings, Settings
from models import RoutePlan, DistanceMatrix, Stop, Bus, Demand, Depot

logger = logging.getLogger(__name__)

class OptimizerService:
    """Service for route optimization using OR-Tools CVRPTW"""

    def __init__(self):
        self.config: Settings = get_settings()
        self._lock = asyncio.Lock()
        self._is_running = False

    async def is_running(self) -> bool:
        """Check if an optimization is currently running"""
        return self._is_running

    def _get_first_solution_strategy(self, strategy_name: str):
        """Map strategy name to OR-Tools FirstSolutionStrategy enum"""
        return getattr(
            routing_enums_pb2.FirstSolutionStrategy,
```

```
            strategy_name,
            routing_enums_pb2.FirstSolutionStrategy.PATH_CHEAPEST_ARC
        )

    def _get_metaheuristic(self, metaheuristic_name: str):
        """Map metaheuristic name to OR-Tools LocalSearchMetaheuristic
enum"""
        return getattr(
            routing_enums_pb2.LocalSearchMetaheuristic,
            metaheuristic_name,
            routing_enums_pb2.LocalSearchMetaheuristic.GUIDED_LOCAL_SE
ARCH
        )

    async def optimize(
        self,
        scenario_type: str,
        matrix_id: UUID | None,
        fuel_cost_per_km: float,
        bus_ids: list[UUID] | None,
        max_ride_time_min: int | None,
        arrival_deadline: str | None,
        enable_split_delivery: bool = True,
    ) -> RoutePlan:
        """
        Run CVRPTW optimization and return stored route plan.

        Algorithm Overview:
        1. Load distance matrix (latest or specified)
        2. Load active stops with demand
        3. Load available buses with depots
        4. Build CVRPTW model with constraints
        5. Solve using OR-Tools
        6. Parse solution into routes
        7. Calculate cost and stats
```

```
        8. Store and return RoutePlan
        """

        # Check if already running
        if self._is_running:
            raise HTTPException(
                status_code=409,
                detail="An optimization is already in progress. Please
wait for it to complete."
            )

        async with self._lock:
            self._is_running = True
            try:
                return await self._do_optimize(
                    scenario_type, matrix_id,
                        fuel_cost_per_km, bus_ids, max_ride_time_min,
arrival_deadline,
                    enable_split_delivery
                )
            finally:
                self._is_running = False

    async def _do_optimize(
        self,
        scenario_type: str,
        matrix_id: UUID | None,
        fuel_cost_per_km: float,
        bus_ids: list[UUID] | None,
        max_ride_time_min: int | None,
        arrival_deadline: str | None,
        enable_split_delivery: bool = True,
    ) -> RoutePlan:
        """Internal optimization logic"""
```

```
        logger.info("=" * 60)
        logger.info("=== OPTIMIZATION STARTED ===")
                logger.info(f"Scenario:   {scenario_type},   Deadline:
{arrival_deadline}")
               logger.info(f"Config:   max_ride={max_ride_time_min   or
self.config.max_ride_time_min}min,    deadline={arrival_deadline    or
self.config.arrival_deadline}, fuel_cost={fuel_cost_per_km}/km")

        try:
            # PHASE 1: Fetch all data (session will close after this)
            async with async_session_maker() as session:
                # Step 1: Load distance matrix
                logger.info("[STEP 1] Loading distance matrix...")
                matrix = await self._load_matrix(session, matrix_id)
                if not matrix:
                   logger.error("No distance matrix found in database")
                    raise HTTPException(
                        status_code=400,
                       detail="No distance matrix available. Build one
first."
                    )

                matrix_data = matrix.matrix_json
                matrix_id = matrix.id
                matrix_stop_count = matrix.stop_count
                depot_ids_in_matrix = matrix_data.get("depot_ids", [])
                depot_names = matrix_data.get("depot_names", {})
                logger.info(f"  Matrix ID: {matrix_id}")
                logger.info(f"  Student stops: {matrix_stop_count}")
                logger.info(f"  Depots: {len(depot_ids_in_matrix)}")

                # Step 2: Load stops with demand
                logger.info("[STEP 2] Loading stops with demand...")
```

```
                                        stops_with_demand    =    await
self._load_stops_with_demand(session, enable_split_delivery)
                if not stops_with_demand:
                  logger.error("No stops with demand found in database")
                    raise HTTPException(
                        status_code=400,
                        detail="No active stops with demand found."
                    )
                    total_students  =  sum(s['student_count']  for  s  in
stops_with_demand)
               logger.info(f"  Stops loaded: {len(stops_with_demand)}")
                logger.info(f"  Total students: {total_students}")

                # Step 3: Load buses
                logger.info("[STEP 3] Loading buses...")
                buses = await self._load_buses(session, bus_ids)
                if not buses:
                    logger.error("No buses found in database")
                    raise HTTPException(
                        status_code=400,
                        detail="No buses available."
                    )
                total_capacity = sum(b['capacity'] for b in buses)
                logger.info(f"  Buses loaded: {len(buses)}")
                logger.info(f"  Total capacity: {total_capacity}")
                for i, bus in enumerate(buses):
                        logger.info(f"     Bus  {i+1}:  {bus['bus_no']},
capacity={bus['capacity']}")

                # Check capacity constraint - FAIL FAST
                if total_students > total_capacity:
                       logger.error(f"CAPACITY  ERROR:  {total_students}
students > {total_capacity} total capacity ({len(buses)} buses)")
                    raise HTTPException(
                        status_code=422,
```

```
                        detail={
                            "error_type": "capacity_insufficient",
                           "message": f"Fleet capacity insufficient to
handle all students",
                            "details": {
                                "total_students": total_students,
                                "total_capacity": total_capacity,
                                "buses_available": len(buses),
                                   "capacity_needed": total_students -
total_capacity,
                             "additional_buses_needed": (total_students
- total_capacity + 49) // 50
                            },
                            "suggestions": [
                                    f"Add at least {(total_students -
total_capacity + 49) // 50} more buses (50 capacity each)",
                                     f"Increase existing bus capacities by
{total_students - total_capacity} total seats",
                                  f"Current: {total_capacity} seats across
{len(buses)} buses",
                                   f"Required: {total_students} seats for
{total_students} students"
                            ]
                        }
                    )

            # PHASE 2: Solve + Process + Save (no DB connection during
solve!)
            return await self._do_solve_and_save(
                matrix_data=matrix_data,
                matrix_id=matrix_id,
                matrix_stop_count=matrix_stop_count,
                stops_with_demand=stops_with_demand,
                buses=buses,
                scenario_type=scenario_type,
```

```
                fuel_cost_per_km=fuel_cost_per_km,
                              max_ride_time_min=max_ride_time_min   or
self.config.max_ride_time_min,
                               arrival_deadline=arrival_deadline   or
self.config.arrival_deadline,
                enable_split_delivery=enable_split_delivery,
            )
        except HTTPException:
            raise
        except Exception as e:
            logger.error("=" * 60)
            logger.error("=== OPTIMIZATION FAILED ===")
            logger.error(f"Error type: {type(e).__name__}")
            logger.error(f"Error message: {str(e)}")
            logger.error(f"Full traceback: {traceback.format_exc()}")
            logger.error("=" * 60)
            raise HTTPException(
                status_code=500,
                detail=f"Optimization failed: {str(e)}"
            )

    async def _load_matrix(
        self,
        session: AsyncSession,
        matrix_id: UUID | None
    ) -> DistanceMatrix | None:
        """Load distance matrix by ID or get latest"""

        if matrix_id:
            result = await session.execute(
                     select(DistanceMatrix).where(DistanceMatrix.id  ==
matrix_id)
            )
            return result.scalar_one_or_none()
        else:
```

```
            result = await session.execute(
                select(DistanceMatrix).order_by(DistanceMatrix.created
_at.desc()).limit(1)
            )
            return result.scalar_one_or_none()

    async def _load_stops_with_demand(
        self,
        session: AsyncSession,
        enable_split_delivery: bool = True,
    ) -> List[Dict[str, Any]]:
        """Load active stops with their demand"""

        query = (
            select(Stop, Demand)
            .join(Demand, Stop.id == Demand.stop_id)
            .where(Stop.active == True)
            .where(Stop.lat.isnot(None))
            .where(Stop.lon.isnot(None))
            .where(Demand.student_count > 0)
        )

        result = await session.execute(query)
        rows = result.all()

        # Aggregate demand by stop to handle potential duplicate demand
records
        stop_demand_agg = {}
        for stop, demand in rows:
            stop_id = str(stop.id)
            if stop_id not in stop_demand_agg:
                stop_demand_agg[stop_id] = {
                    "id": stop_id,
                    "name": stop.name,
                    "code": stop.stop_code,
```

```
                    "lat": stop.lat,
                    "lon": stop.lon,
                    "zone": stop.zone,
                    "student_count": 0
                }
                        stop_demand_agg[stop_id]["student_count"]   +=
demand.student_count

        stops_with_demand = list(stop_demand_agg.values())

             logger.info(f"   Stops   with   positive   demand   loaded:
{len(stops_with_demand)}")

        if enable_split_delivery:
                                                stops_with_demand      =
self._split_stop_demand(stops_with_demand)

        return stops_with_demand

       def   _split_stop_demand(self,   stops:   List[Dict[str,   Any]],
enable_split_delivery: bool = True) -> List[Dict[str, Any]]:
          """Split stop demand into multiple entries for better bus
packing.

            If  a  stop  has  50+  students,  split  into  chunks  of
max_bus_capacity.
         This allows split delivery - students from same stop can use
multiple buses.
        """
        if not enable_split_delivery:
            return stops

        max_capacity = self.config.max_bus_capacity
        split_stops = []
```

```
        for stop in stops:
            demand = stop["student_count"]
            if demand > max_capacity:
               num_splits = (demand + max_capacity - 1) // max_capacity
                chunk_size = max_capacity
                for i in range(num_splits):
                    remaining = demand - (i * chunk_size)
                    actual_chunk = min(chunk_size, remaining)
                    split_stops.append({
                        **stop,
                        "id": f"{stop['id']}_split_{i}",
                        "name": f"{stop['name']} (Part {i+1})",
                        "student_count": actual_chunk,
                        "is_split": True,
                        "parent_stop_id": stop["id"]
                    })
            else:
                split_stops.append(stop)

        total_original = len(stops)
        total_split = len(split_stops)
           logger.info(f"  Split  delivery:  {total_original}  stops  →
{total_split} split nodes (max {max_capacity} per node)")

        return split_stops

    async def _load_buses(
        self,
        session: AsyncSession,
        bus_ids: list[UUID] | None
    ) -> List[Dict[str, Any]]:
        """Load available buses with depot info"""

        query = select(Bus).options(selectinload(Bus.depot))
```

```
        if bus_ids:
            query = query.where(Bus.id.in_(bus_ids))

        result = await session.execute(query)
        buses = result.scalars().all()

        buses_list = []
        for bus in buses:
            bus_data = {
                "id": str(bus.id),
                "bus_no": bus.bus_no,
                                        "capacity":    bus.capacity    or
self.config.max_bus_capacity,
               "depot_id": str(bus.depot_id) if bus.depot_id else None,
                "depot_lat": bus.depot.lat if bus.depot else None,
                "depot_lon": bus.depot.lon if bus.depot else None,
            }
            buses_list.append(bus_data)

        return buses_list

    async def _do_solve_and_save(
        self,
        matrix_data: Dict[str, Any],
        matrix_id: UUID,
        matrix_stop_count: int,
        stops_with_demand: List[Dict],
        buses: List[Dict],
        scenario_type: str,
        fuel_cost_per_km: float,
        max_ride_time_min: int,
        arrival_deadline: str,
        enable_split_delivery: bool = True,
    ) -> RoutePlan:
```

```
        """PHASE 2: Solve + Process + Save (no DB connection during 1-
hour solve!)"""

        try:
                 total_students  =  sum(s['student_count']  for  s  in
stops_with_demand)

            logger.info("=" * 60)
           logger.info("=== PHASE 2: SOLVE (DB connection closed) ===")
           logger.info(f"  Solving with {len(stops_with_demand)} stops,
{len(buses)} buses")
                                          logger.info(f"       Timeout:
{self.config.optimization_timeout_sec}s")

            # PHASE 2a: Solve (no DB connection!)
            t0 = time.time()
            preliminary_routes, valid_stops = await asyncio.to_thread(
                self._solve_cvrptw,
                matrix_data=matrix_data,
                stops_with_demand=stops_with_demand,
                buses=buses,
                campus_lat=self.config.campus_lat,
                campus_lon=self.config.campus_lon,
                max_ride_time=max_ride_time_min,
                arrival_deadline=arrival_deadline,
                timeout=self.config.optimization_timeout_sec,
            )

            # PHASE 2b: Process road geometries (no DB connection)
                logger.info("[STEP  5]  Fetching  road  geometries  and
calculating accurate timings...")
            routes, stats = await self._process_road_accurate_routes(
                preliminary_routes=preliminary_routes,
                valid_stops=valid_stops,
                campus_lat=self.config.campus_lat,
```

```
                campus_lon=self.config.campus_lon,
                arrival_deadline=arrival_deadline,
                max_ride_time=max_ride_time_min,
                enable_split_delivery=enable_split_delivery
            )
            model_build_time = time.time() - t0

            logger.info(f"  Routes generated: {len(routes)}")
                                logger.info(f"     Students    assigned:
{stats.get('total_students_assigned', 0)}/{total_students}")
           logger.info(f"  Coverage: {stats.get('coverage_percentage',
0):.1f}%")
           logger.info(f"  Total solve time: {model_build_time:.2f}s")

            # PHASE 2c: Calculate cost
           total_distance = sum(r["total_distance_km"] for r in routes)
            cost_estimate = total_distance * fuel_cost_per_km

            logger.info(f"[STEP 6] Calculating cost")
            logger.info(f"  Total distance: {total_distance}km")
            logger.info(f"  Cost estimate: ₹{cost_estimate:.2f}")

            # PHASE 2d: Save to DB (fresh connection!)
            logger.info("[STEP 7] Storing route plan in database...")

            async with async_session_maker() as save_session:
                route_plan = RoutePlan(
                    scenario_type=scenario_type,
                    routes_json={"routes": routes},
                    stats_json={
                        **stats,
                        "solve_time_seconds": model_build_time,
                        "model_build_time_seconds": model_build_time,
                    },
                    cost_estimate=cost_estimate,
```

```
                )
                save_session.add(route_plan)
                await save_session.commit()
                await save_session.refresh(route_plan)

            logger.info(f"  Route Plan ID: {route_plan.id}")
            logger.info("=== OPTIMIZATION COMPLETED SUCCESSFULLY ===")
            logger.info("=" * 60)

            return route_plan

        except HTTPException:
            raise
        except Exception as e:
            logger.error("=" * 60)
            logger.error("=== SOLVE FAILED ===")
            logger.error(f"Error type: {type(e).__name__}")
            logger.error(f"Error message: {str(e)}")
            logger.error(f"Full traceback: {traceback.format_exc()}")
            logger.error("=" * 60)
            raise HTTPException(
                status_code=500,
                detail=f"Optimization failed: {str(e)}"
            )

    def _solve_cvrptw(
        self,
        matrix_data: Dict[str, Any],
        stops_with_demand: List[Dict],
        buses: List[Dict],
        campus_lat: float,
        campus_lon: float,
        max_ride_time: int,
        arrival_deadline: str,
        timeout: int,
```

```
    ) -> Tuple[List[Dict], Dict[str, Any]]:
        """
        Build and solve CVRPTW model using OR-Tools - Integrated Multi-
Depot Approach.

        Node Mapping:
        - Node 0: Virtual Source (Start for all vehicles)
        - Node 1: Campus (End for all vehicles)
        - Nodes 2 to D+1: Real Depots
        - Nodes D+2 to N: Student Stops
        """

        logger.info("=" * 60)
        logger.info("[CVRPTW] Starting solver setup - INTEGRATED MULTI-
DEPOT")

        # Parse matrix data
        stop_ids_in_matrix = matrix_data["stop_ids"]
        depot_ids_in_matrix = matrix_data.get("depot_ids", [])
        depot_names = matrix_data.get("depot_names", {})
        depot_coords = matrix_data.get("depot_coords", {})
        durations = matrix_data["durations"]  # seconds
        distances = matrix_data["distances"]  # meters

        # OSRM matrix mapping (0=Campus, 1..N=Stops, N+1..N+D=Depots)
        matrix_campus_idx = matrix_data.get("campus_index", 0)

        # Filter stops that exist in matrix
        valid_stops = [
            s for s in stops_with_demand
            if s["id"] in stop_ids_in_matrix
        ]

        if not valid_stops:
            logger.error("No stops found in distance matrix")
```

```
            raise HTTPException(
                status_code=400,
                  detail="No stops found in distance matrix. Rebuild
matrix with current stops."
            )

        num_stops = len(valid_stops)

        # Node Mapping - REMOVED PHYSICAL DEPOTS AS NODES
        # 0: Virtual Source (Pool of all 250 buses)
        # 1: Campus
        # 2 to 2+S-1: Student Stops
        VIRTUAL_SOURCE = 0
        CAMPUS_NODE = 1
        STOP_START_NODE = 2
        num_nodes = 2 + num_stops

        logger.info(f"  Nodes: {num_nodes} (1 Virtual Start, 1 Campus,
{num_stops} Stops)")

        # Build node translation helpers
            stop_id_to_matrix_idx = {sid: i + 1 for i, sid in
enumerate(stop_ids_in_matrix)}
        # Depots start after all stops in the matrix
        depot_id_to_matrix_idx = {did: i + 1 + len(stop_ids_in_matrix)
for i, did in enumerate(depot_ids_in_matrix)}

        # Mapping for result reconstruction
        index_to_stop = {}
        for i, stop in enumerate(valid_stops):
            index_to_stop[STOP_START_NODE + i] = {
                **stop,
                "matrix_idx": stop_id_to_matrix_idx[stop["id"]]
            }
```

```
        # Physical depot info for choosing the best start
        physical_depots = []
        for did in depot_ids_in_matrix:
            coords = depot_coords.get(did, {})
            physical_depots.append({
                "id": did,
                "name": depot_names.get(did, "Unknown Depot"),
                "lat": coords.get("lat"),
                "lon": coords.get("lon"),
                "matrix_idx": depot_id_to_matrix_idx[did]
            })

        # Build matrices
         logger.info(f"  Building optimized matrices for {num_nodes}
nodes using NumPy...")
        t_matrix_start = time.time()

        PENALTY = 10000000
         time_np = np.full((num_nodes, num_nodes), PENALTY // 1000,
dtype=np.int32)
             cost_np = np.full((num_nodes, num_nodes), PENALTY,
dtype=np.int32)

        durations_np = np.array(durations)
        distances_np = np.array(distances)

        # OSRM matrix indices
        stop_matrix_indices = [index_to_stop[i]["matrix_idx"] for i in
range(STOP_START_NODE, num_nodes)]
             depot_matrix_indices = [d["matrix_idx"] for d in
physical_depots]

        # 1. Virtual Source -> Stops (INFINITE DEPOT LOGIC)
        # The cost to start at a stop is the distance from the CLOSEST
physical depot to that stop.
```

```
        if depot_matrix_indices:
              # Get durations from all depots to all stops: shape
(num_depots, num_stops)
          depot_to_stop_dur = durations_np[depot_matrix_indices, :][:,
stop_matrix_indices]
              depot_to_stop_dist = distances_np[depot_matrix_indices,
:][:, stop_matrix_indices]

            # Use the minimum (closest depot) for each stop
                        time_np[VIRTUAL_SOURCE, STOP_START_NODE:] =
(np.min(depot_to_stop_dur, axis=0) / 60).astype(np.int32)
                        cost_np[VIRTUAL_SOURCE, STOP_START_NODE:] =
np.min(depot_to_stop_dist, axis=0).astype(np.int32)

        # 2. Virtual Source -> Campus (Escape hatch)
        time_np[VIRTUAL_SOURCE, CAMPUS_NODE] = 0
        cost_np[VIRTUAL_SOURCE, CAMPUS_NODE] = 0

        # 3. Stops -> Stops
        for i, mi in enumerate(stop_matrix_indices):
            solver_idx_i = STOP_START_NODE + i
            valid_mask = durations_np[mi, stop_matrix_indices] >= 0
                time_np[solver_idx_i, np.array(range(STOP_START_NODE,
num_nodes))[valid_mask]] = (durations_np[mi,
stop_matrix_indices][valid_mask] / 60).astype(np.int32)
                cost_np[solver_idx_i, np.array(range(STOP_START_NODE,
num_nodes))[valid_mask]] = distances_np[mi,
stop_matrix_indices][valid_mask].astype(np.int32)

        # 4. Stops -> Campus
        for i, mi in enumerate(stop_matrix_indices):
            solver_idx_i = STOP_START_NODE + i
            if durations_np[mi, matrix_campus_idx] >= 0:
                              time_np[solver_idx_i, CAMPUS_NODE] =
int(durations_np[mi, matrix_campus_idx] / 60)
```

```
                                cost_np[solver_idx_i,   CAMPUS_NODE]   =
int(distances_np[mi, matrix_campus_idx])

        # 5. Explicit Self-loops
        for i in range(num_nodes):
            time_np[i, i] = 0
            cost_np[i, i] = 0

        # Convert to list for OR-Tools compatibility
        time_matrix = time_np.tolist()
        cost_matrix = cost_np.tolist()

        matrix_build_time = time.time() - t_matrix_start
         logger.info(f"  Matrices  built  in  {matrix_build_time:.2f}s.
Initializing Routing Manager...")

        # Demands
        demands = [0] * num_nodes
        for idx, stop in index_to_stop.items():
            demands[idx] = stop["student_count"]

        # Vehicle Data
        num_vehicles = len(buses)
        vehicle_capacities = [b["capacity"] for b in buses]
        starts = [VIRTUAL_SOURCE] * num_vehicles
        ends = [CAMPUS_NODE] * num_vehicles

        t_manager_start = time.time()
       manager = pywrapcp.RoutingIndexManager(num_nodes, num_vehicles,
starts, ends)
        routing = pywrapcp.RoutingModel(manager)
           logger.info(f"  Routing  Model  created  in  {time.time()  -
t_manager_start:.2f}s. Pre-calculating index mappings...")
```

```
        # OPTIMIZATION: Pre-calculate index-to-node mapping using the
correct index count
            index_to_node_map = [manager.IndexToNode(i) for i in
range(manager.GetNumberOfIndices())]

        # Callbacks (Optimized with pre-calculated mapping)
        def cost_callback(from_index, to_index):
            from_node = index_to_node_map[from_index]
            to_node = index_to_node_map[to_index]
            return cost_matrix[from_node][to_node]

        def time_callback(from_index, to_index):
            from_node = index_to_node_map[from_index]
            to_node = index_to_node_map[to_index]
            return time_matrix[from_node][to_node]

        cost_idx = routing.RegisterTransitCallback(cost_callback)
        time_idx = routing.RegisterTransitCallback(time_callback)

        routing.SetArcCostEvaluatorOfAllVehicles(cost_idx)

        # Dimensions
        logger.info("  Adding dimensions...")
                                        routing.AddDimension(time_idx,
self.config.time_dimension_slack, max_ride_time, False, "Time")
        time_dimension = routing.GetDimensionOrDie("Time")
        deadline_min = self._time_to_minutes(arrival_deadline)

        # Apply deadline to each vehicle's end node
        for i in range(num_vehicles):
                    time_dimension.CumulVar(routing.End(i)).SetRange(0,
deadline_min)

        def demand_callback(from_index):
            return demands[index_to_node_map[from_index]]
```

```
                                                demand_idx           =
routing.RegisterUnaryTransitCallback(demand_callback)
              routing.AddDimensionWithVehicleCapacity(demand_idx,   0,
vehicle_capacities, True, "Capacity")

        routing.SetFixedCostOfAllVehicles(self.config.fixed_vehicle_co
st)

        # Allow dropping stops if they cause infeasibility
        drop_penalty = self.config.drop_penalty
        for i in range(STOP_START_NODE, num_nodes):
                      routing.AddDisjunction([manager.NodeToIndex(i)],
drop_penalty)

        # Search parameters
        search_params = pywrapcp.DefaultRoutingSearchParameters()
                           search_params.first_solution_strategy     =
self._get_first_solution_strategy(self.config.first_solution_strategy)
                         search_params.local_search_metaheuristic     =
self._get_metaheuristic(self.config.local_search_metaheuristic)
        search_params.time_limit.FromSeconds(timeout)

           logger.info(f"  Model  fully  initialized.  Starting  solver
(timeout {timeout}s)...")
        solve_start = time.time()
        solution = routing.SolveWithParameters(search_params)
        solve_time = time.time() - solve_start
        logger.info(f"  Solver finished in {solve_time:.2f}s")

        if not solution:
                logger.error("Solver  returned  no  solution  even  with
disjunctions!")
              raise HTTPException(status_code=422, detail="No feasible
solution found.")
```

```
        # Process routes
        preliminary_routes = []
        assigned_stops_count = 0
        for vehicle_id in range(num_vehicles):
            index = routing.Start(vehicle_id)
            route_nodes = []
            while not routing.IsEnd(index):
                node_idx = manager.IndexToNode(index)
                route_nodes.append(node_idx)
                index = solution.Value(routing.NextVar(index))

            # Extract Depot and Stops (Skip Virtual Source)
            # Route nodes: [VirtualSource, Stop1, Stop2...]
            if len(route_nodes) > 1: # Has at least one Stop
                stop_nodes = route_nodes[1:]
                route_stops = [index_to_stop[sn] for sn in stop_nodes]

                 # RECONSTRUCT DEPOT CHOICE: Which depot was actually
closest to the first stop?
                first_stop = route_stops[0]
                first_stop_matrix_idx = first_stop["matrix_idx"]

                best_depot = physical_depots[0]
                min_dist = float('inf')
                for depot in physical_depots:
                                                                d     =
distances_np[depot["matrix_idx"]][first_stop_matrix_idx]
                    if d < min_dist:
                        min_dist = d
                        best_depot = depot

                preliminary_routes.append({
                    "bus": buses[vehicle_id],
                    "depot": best_depot,
                    "stops": route_stops
```

```
                })
                assignment_stops_count += len(stop_nodes)

                        logger.info(f"      Assignment     complete:
{assigned_stops_count}/{num_stops} stops assigned.")
        if assigned_stops_count < num_stops:
            logger.warning(f"  {num_stops - assigned_stops_count} stops
were dropped due to constraints.")

        return preliminary_routes, valid_stops

    async def _process_road_accurate_routes(
        self,
        preliminary_routes: List[Dict],
        valid_stops: List[Dict],
        campus_lat: float,
        campus_lon: float,
        arrival_deadline: str,
        max_ride_time: int,
        enable_split_delivery: bool = True,
    ) -> Tuple[List[Dict], Dict]:
        """Async processing of routes to get real road geometries and
timings from OSRM"""
        from services.osrm import osrm_service

        async def process_single_route(route_info):
            bus = route_info["bus"]
            stops = route_info["stops"]
            depot = route_info["depot"]

               coords = [(depot["lat"], depot["lon"])] + [(s["lat"],
s["lon"]) for s in stops] + [(campus_lat, campus_lon)]

            try:
                osrm_data = await osrm_service.get_route(coords)
```

```
                osrm_route = osrm_data["routes"][0]
                            geometry  =  [[p[1],  p[0]]  for  p  in
osrm_route["geometry"]["coordinates"]]
                legs = osrm_route["legs"]

                deadline_min = self._time_to_minutes(arrival_deadline)
                total_time_min = round(osrm_route["duration"] / 60)
                start_time_min = deadline_min - total_time_min
                current_time_min = deadline_min
                processed_stops = []

                # Backward pass for arrival times
                for i in range(len(stops) - 1, -1, -1):
                    leg_to_next = legs[i + 1]
                     current_time_min -= round(leg_to_next["duration"]
/ 60)

                    stop = stops[i]
                    processed_stops.append({
                        "stop_id": stop["id"],
                        "stop_name": stop["name"],
                        "stop_code": stop.get("code"),
                        "lat": stop["lat"],
                        "lon": stop["lon"],
                        "zone": stop.get("zone"),
                                                "arrival_time":
self._minutes_to_time(current_time_min),
                        "students_boarding": stop["student_count"],
                            "cumulative_time_min": current_time_min -
start_time_min,
                                            "distance_from_prev_km":
round(legs[i]["distance"] / 1000, 2),
                        "parent_stop_id": stop.get("parent_stop_id"),
                        "is_split": stop.get("is_split", False),
                    })
```

```
                processed_stops.reverse()
                total_dist = osrm_route["distance"] / 1000
                total_time = total_time_min
                students = sum(s["student_count"] for s in stops)
                utilization = (students / bus["capacity"]) * 100

                warnings = []
                if total_time > max_ride_time:
                  warnings.append(f"Exceeds max ride time ({total_time}
min)")
                if utilization > 95:
                          warnings.append(f"High capacity utilization
({utilization:.1f}%)")

                return {
                    "bus_id": bus["id"],
                    "bus_no": bus["bus_no"],
                    "capacity": bus["capacity"],
                    "depot_id": str(depot["id"]),
                    "depot_name": depot["name"],
                    "depot_lat": depot["lat"],
                    "depot_lon": depot["lon"],
                    "stops": processed_stops,
                    "geometry": geometry,
                    "total_students": students,
                    "total_distance_km": round(total_dist, 2),
                    "total_time_min": total_time,
                    "capacity_utilization": round(utilization, 2),
                    "warnings": warnings,
                }
            except Exception as e:
                      logger.error(f"Error processing route for bus
{bus['bus_no']}: {str(e)}")
                return None
```

```
        # Execute in parallel
        routes = await asyncio.gather(*(process_single_route(r) for r
in preliminary_routes))
        routes = [r for r in routes if r is not None]

        # Calculate final stats
        total_assigned = sum(r["total_students"] for r in routes)

        # Handle split delivery: get original stop counts
        if enable_split_delivery:
                original_stops = [s for s in valid_stops if not
s.get("is_split", False)]
                total_students_req = sum(s["student_count"] for s in
original_stops)
            assigned_parent_ids = set()
            for r in routes:
                for s in r["stops"]:
                    parent_id = s.get("parent_stop_id", s["stop_id"])
                    assigned_parent_ids.add(parent_id)

            unassigned = []
            for s in original_stops:
                if s["id"] not in assigned_parent_ids:
                        unassigned.append({"stop_id": s["id"], "name":
s["name"], "reason": "Constraints", "lat": s["lat"], "lon": s["lon"]})
        else:
                total_students_req = sum(s["student_count"] for s in
valid_stops)
            assigned_stop_ids = {s["stop_id"] for r in routes for s in
r["stops"]}

            unassigned = []
            for s in valid_stops:
                if s["id"] not in assigned_stop_ids:
```

```
                        unassigned.append({"stop_id": s["id"], "name":
s["name"], "reason": "Constraints", "lat": s["lat"], "lon": s["lon"]})

        stats = {
            "total_buses_used": len(routes),
             "total_distance_km": sum(r["total_distance_km"] for r in
routes),
            "total_time_min": max(r["total_time_min"] for r in routes)
if routes else 0,
             "avg_utilization": sum(r["capacity_utilization"] for r in
routes) / len(routes) if routes else 0,
            "total_students_assigned": total_assigned,
            "total_students_requested": total_students_req,
           "coverage_percentage": (total_assigned / total_students_req
* 100) if total_students_req > 0 else 0,
            "unassigned_stops": unassigned,
                     "global_warnings":  [f"Bus  {r['bus_no']}:  {',
'.join(r['warnings'])}" for r in routes if r["warnings"]],
        }

        return routes, stats

    def _time_to_minutes(self, time_str: str) -> int:
        """Convert HH:MM to minutes from midnight"""
        parts = time_str.split(":")
        return int(parts[0]) * 60 + int(parts[1])

    def _minutes_to_time(self, minutes: int) -> str:
        """Convert minutes from midnight to HH:MM"""
        hours = minutes // 60
        mins = minutes % 60
        return f"{hours:02d}:{mins:02d}"

# Singleton instance
optimizer_service = OptimizerService()
```

4.3 Discussion of any challenges faced during implementation and their solutions

Challenge 1: Unrealistic Algorithmic Outputs (The Physics Problem)

- *Issue:* Initial tests of the VRP solver produced mathematically perfect routes that were physically impossible to drive in cities like Gurugram or Faridabad (e.g., suggesting a 50-seater bus make a sharp U-turn on a busy highway to save 2 minutes).
- *Solution:* We solved this by diving into the Open Source Routing Machine's core logic. We engineered a custom Lua profile (bus.lua) that injected heavy time penalties (40+ seconds) for U-turns and capped speeds based on road classifications, forcing the algorithm to prefer main arterial roads and logical loops.

Challenge 2: The "Local Minima" Trap in Optimization

- *Issue:* When processing datasets exceeding 3,000 students, the default "Savings" heuristic in OR-Tools failed to efficiently pack the buses. It would get stuck in a "local minimum" (a good route, but not the best overall fleet plan), resulting in more buses being deployed than necessary.
- *Solution:* We benchmarked over 30 solver configurations. The breakthrough was replacing the standard heuristic with the **Christofides** algorithm for the initial graph, and subsequently applying **Simulated Annealing** as the local search metaheuristic. This combination successfully optimized the fleet, reducing required bus usage significantly.

Challenge 3: Browser Performance and Map Rendering Crashes

- *Issue:* Once the backend successfully generated the routes for 40+ buses, transmitting the raw geographical data (GeoJSON) to the Next.js frontend caused severe memory leaks. The browser would freeze or crash when attempting to render tens of thousands of coordinate polylines via Leaflet.
- *Solution:* We modernized the frontend architecture by transitioning from standard GeoJSON to **PMTiles**. By storing and serving the map data as cloud-optimized, edge-rendered vector tiles, the browser only loads the coordinates currently visible in the viewport, resulting in a seamless, lag-free dashboard.

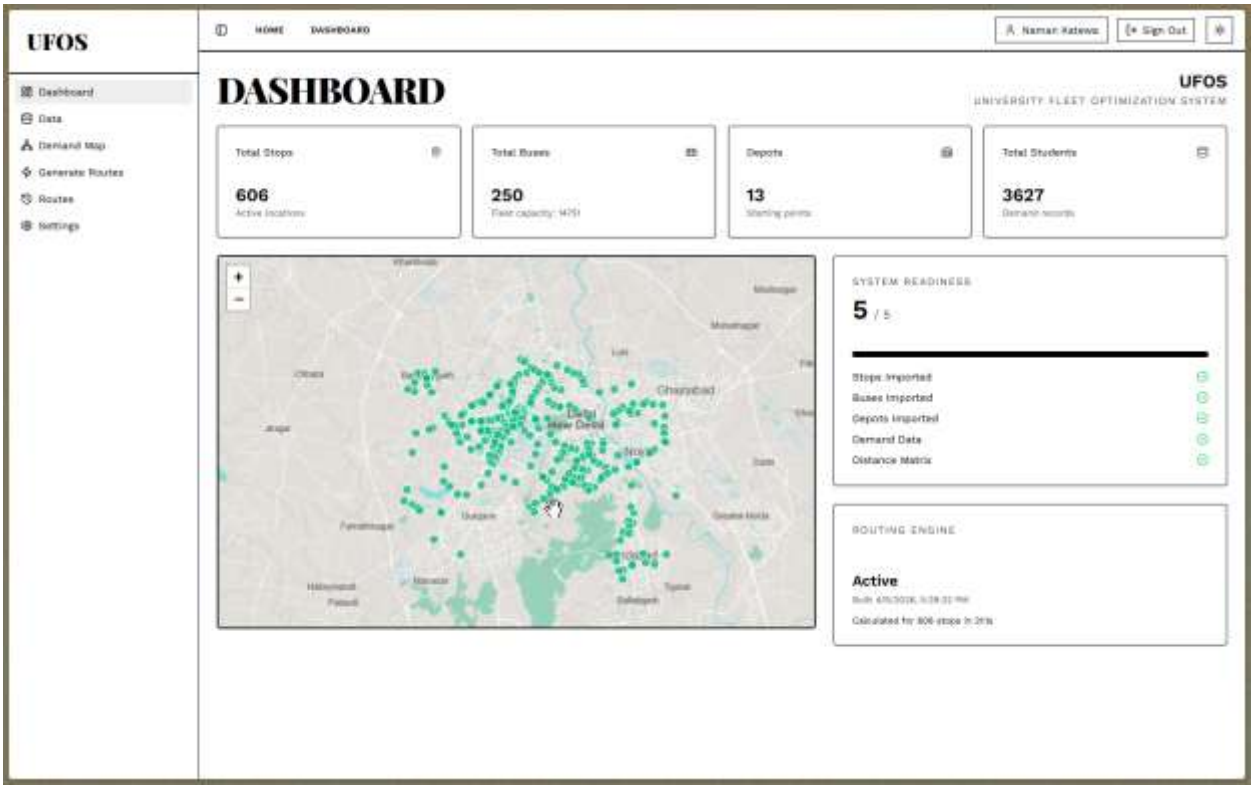
Challenge 4: Production Security and Cross-Origin Restrictions

- *Issue:* Deploying a decoupled architecture (Frontend on Vercel, Backend and Routing Engine on a local VPS) resulted in strict browser CORS (Cross-Origin Resource Sharing) blockages and "Mixed Content" errors due to the lack of SSL on the self-hosted backend.
- *Solution:* An automated deployment pipeline was established. We configured an **Nginx Reverse Proxy** to handle requests safely and utilized **DuckDNS** paired with **Let's Encrypt** to generate valid SSL certificates for the VPS. This established a secure HTTPS tunnel between the Next.js dashboard and the FastAPI backend, resolving all CORS and caching issues.

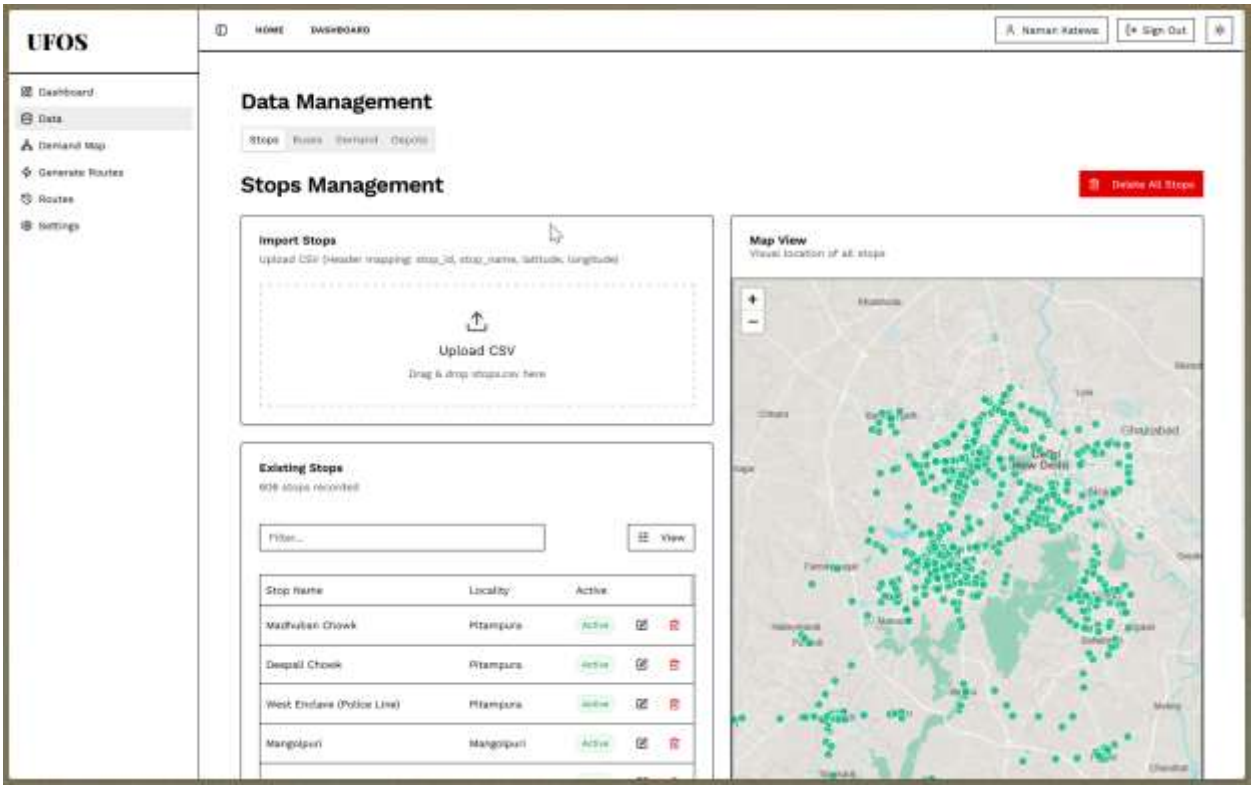
Chapter 5

RESULTS AND DISCUSSIONS

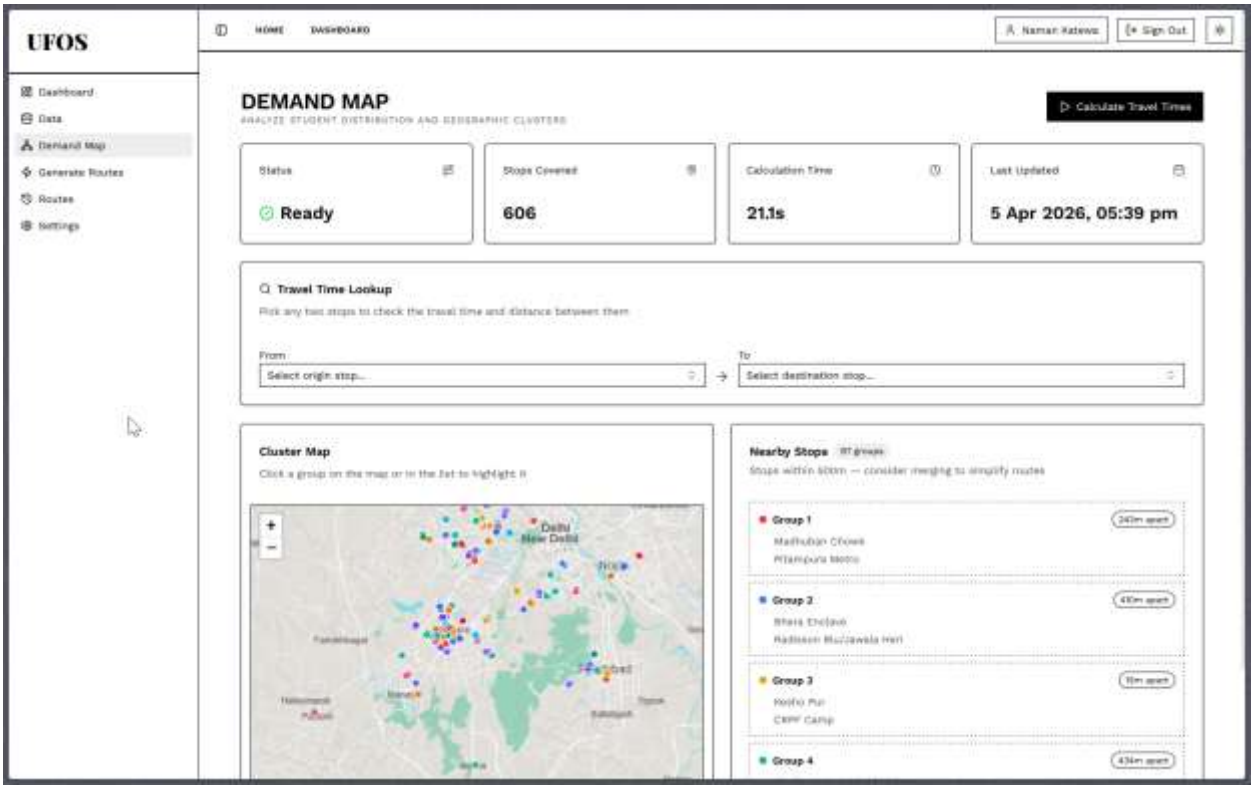
DASHBOARD:



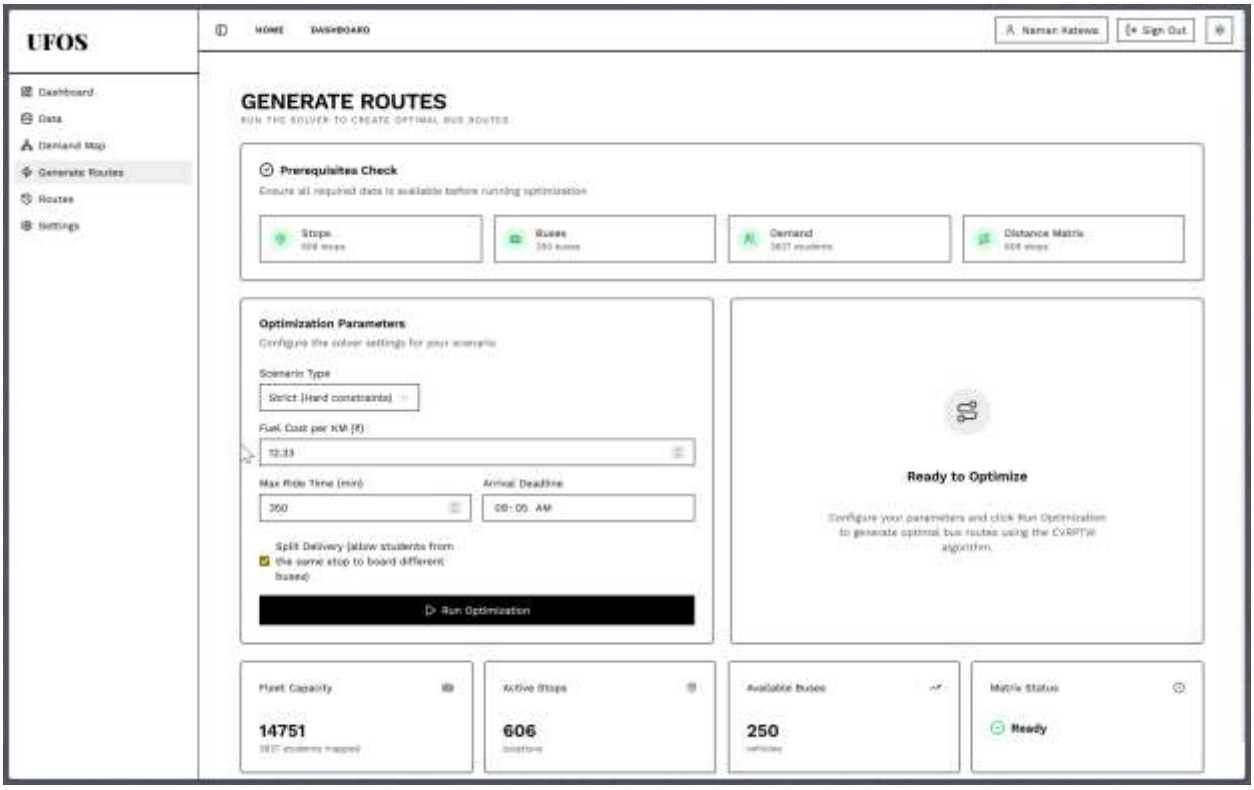
DATA MANAGEMENT:



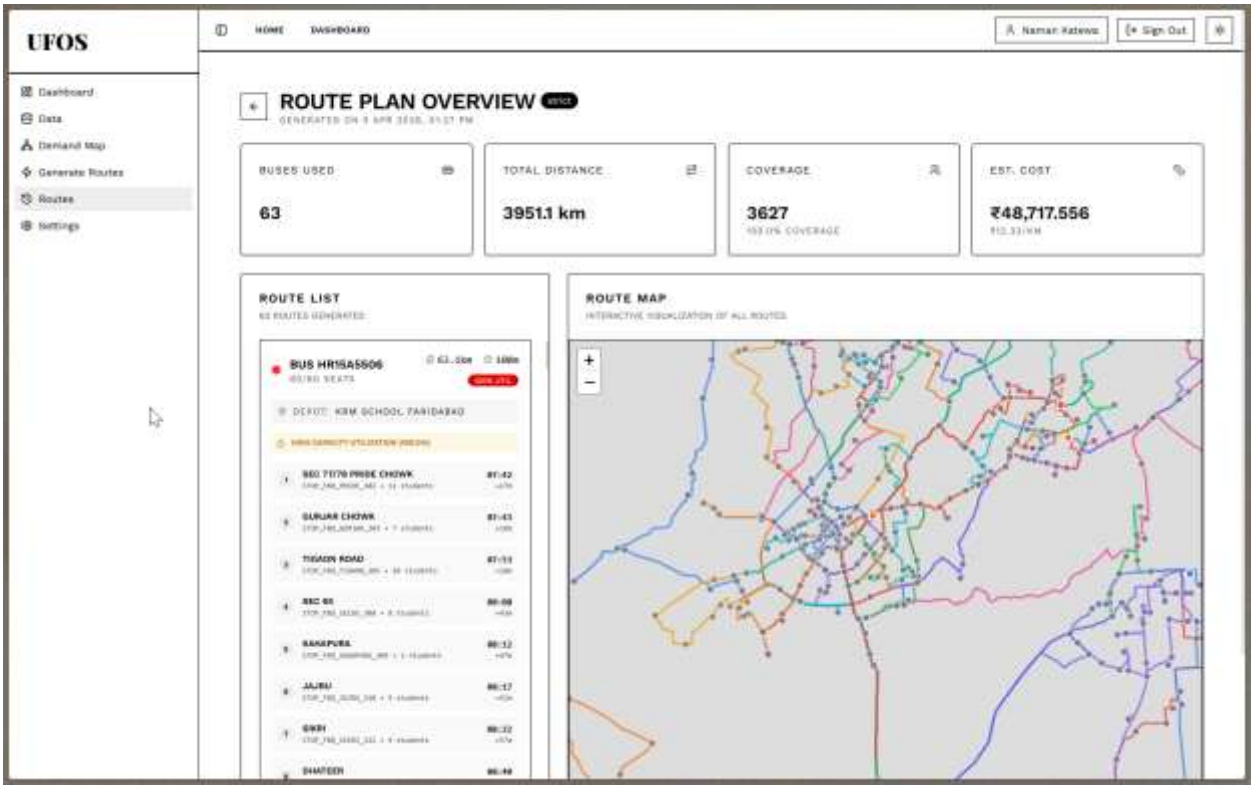
DEMAND MAP (MATRIX GENERATION):



ROUTE GENERATION:



GENERATED ROUTES:



Chapter 6

FUTURE WORK

The University Fleet Optimization System (UFOS) is currently functioning exceptionally well for its primary objectives: generating zero-cost, physics-aware, and mathematically optimized static routes for the extended NCR catchment area. The integration of self-hosted OSRM with Google OR-Tools successfully resolves the Multi-Depot Capacitated Vehicle Routing Problem with Time Windows (MD-CVRPTW), and the PMTiles-backed dashboard provides seamless visualization. However, to evolve the system from a static planning tool into a comprehensive, real-time logistics platform, several avenues for future work can be explored:

1. Real-Time GPS Tracking and Dynamic Re-Routing Currently, UFOS is designed for static semester planning. Future work could include integrating IoT-based GPS tracking hardware installed on the university buses. By feeding live telematics data back to the FastAPI backend, the system could provide real-time ETAs to transport administrators. Furthermore, the solver could be upgraded to handle dynamic re-routing, automatically suggesting alternative paths if a bus encounters unexpected traffic congestion or road closures in the NCR region.

2. Machine Learning for Predictive Demand Forecasting The current optimization relies on maximum student capacity data (e.g., assuming 100% attendance for a given stop). Future enhancements could integrate Machine Learning algorithms (such as XGBoost or LSTM time-series models) to analyze historical attendance, weather patterns, and examination schedules. This would allow the system to predict actual daily passenger demand, enabling administrators to downsize the deployed fleet on low-turnout days, thereby saving even more fuel.

3. Student-Facing Mobile Application While the current project focuses on a Next.js dashboard for administrators, the ecosystem could be expanded to include a cross-platform mobile application for the students. This application could allow students to track their assigned bus in real-time, receive delay notifications, and dynamically "opt-in" or "opt-out" of transport for the day. This live feedback loop would further refine the backend optimization engine.

4. RFID/Biometric Integration for Capacity Validation To validate the effectiveness of the generated routes, the system could be integrated with RFID or biometric scanners at the bus doors. This would track the exact number of students boarding and alighting at each stop. Comparing this real-world boarding data against the UFOS-predicted loads would allow for continuous algorithmic refinement and ensure strict adherence to seating capacity constraints.

5. Algorithmic Scaling and Distributed Computing While the Christofides and Simulated Annealing metaheuristics efficiently process the current load of ~3,000 students, scaling the system to manage larger university networks (e.g., 10,000+ nodes) will increase computation time. Future iterations could implement distributed computing using task queues (like Celery and Redis) or explore GPU-accelerated solvers to reduce processing times from minutes to mere seconds.

Chapter 7

CONCLUSION

In recent years, the logistical demands on educational institutions have grown exponentially. Rapid campus expansions, highly dispersed student populations across diverse urban and semi-urban topographies, and increasing traffic congestion have made university fleet management incredibly complex. Furthermore, the rising costs of fuel and the strict budget constraints of educational institutions have made traditional, manual transportation planning both financially and operationally unsustainable.

While algorithmic fleet management systems are standard tools used to solve these logistical challenges in the corporate sector, the traditional approaches available to universities have severe flaws. Institutions are forced to choose between highly inefficient manual spreadsheet planning or adopting commercial enterprise software. These commercial solutions rely on proprietary mapping algorithms and paid APIs (such as Google Maps), introducing exorbitant recurring licensing fees that drain university resources and offset any fuel savings gained from optimization.

This project, the University Fleet Optimization System (UFOS), successfully implemented advanced mathematical algorithms and open-source geospatial technologies to overcome the limitations of both manual planning and paid commercial software. By developing a strictly "Zero-Operational Cost" architecture, the system enhanced traditional fleet management by delivering the following core functionalities:

1. **Zero-Cost Matrix Generation:** Utilizing a self-hosted Open Source Routing Machine (OSRM) to calculate massive real-world travel-time matrices locally, bypassing paid APIs.

2. **Physics-Aware Routing:** Engineering custom Lua profiles to enforce heavy-bus driving physics, ensuring mathematically generated routes do not suggest impossible U-turns or illegal road usage.
3. **Mathematical Optimization:** Applying Google OR-Tools (Christofides heuristic paired with Simulated Annealing) to solve the Multi-Depot Capacitated Vehicle Routing Problem (MD-CVRPTW), strictly enforcing 50-seat capacities and 120-minute ride limits.
4. **High-Performance Visualization:** Decoupling the frontend and utilizing PMTiles within a Next.js environment to render thousands of route coordinates seamlessly without browser lag.

Ultimately, UFOS provides a robust, enterprise-grade logistics platform that empowers transport administrators to transition from intuition-based scheduling to data-driven optimization. It successfully proves that high-end deep-tech optimization can be democratized for the education sector. The system not only completely eliminates software licensing costs but also guarantees optimized routes that reduce fuel consumption by 10-15%, ensuring equitable, safe, and efficient transportation for students across the entire extended NCR region.

ANNEXURE I:

Plagiarism Declaration Certificate

Title of Work: UFOS (University Fleet Optimization System)
Submitted By: Team OutKast
Institution: K. R. Mangalam University, Gurugram
Department: SOET
Date of Submission: 14/04/2026

I hereby declare that the work entitled **"UFOS"** submitted for academic evaluation and research purposes is my original work. I confirm that:

- I have acknowledged and properly cited all sources, references, and data included in this work.
- This work does not contain any material previously published, written, or prepared by another person, except where due acknowledgment has been made.
- I understand that plagiarism is an academic offense and a violation of research ethics. Any breach of this declaration may lead to disciplinary action as per university policies.
- I have used appropriate referencing techniques and maintained academic integrity throughout this work.

I affirm that the submitted work has normal plagiarism less than 10% and free from AI content.

Signature of Student: _____
Date: 14/04/2026

ANNEXURE II:

Plagiarism Report

- **Generated using software like Turnitin or similar**
- **Shows similarity percentage (<15%) Plagiarism must be less than 15 % for complete report**
- **It will act as proof that the project work was checked**